

FULL-STACK DEVELOPMENT SERIES

Session 4:

Search & Filter

Managing Interactive State & Hook Workflows in React

Lecturer: Msc. Luong Tran Hy Hien

Session Objectives



State Management

Understand how React governs dynamic component memory and lifecycle synchronizations utilizing standard build-in Hooks.



Interactive Querying

Develop responsive search indexes, strict category classifications, and real-time pricing sorters across structural tables.

THEORETICAL FOUNDATION I

React Hooks & Memory

Enabling state storage and event synchronization inside functional React components

Understanding React Hooks



Local State Memory

Functional components natively run state-free. Hooks provide designated hooks into React's background engine memory nodes.



Synchronized Cycles

Hooks streamline resource tracking, event subscription cleanups, and external REST API handshakes sequentially.



Hook Enforcements

Hooks must strictly deploy at top component levels and are prohibited within loops, conditions, or nested structures.

useState: Component State



Preserving Variable Contexts

The useState hook yields a state variable coupled to an accessor method that triggers DOM re-renders upon mutation.

- ▶ **Initial State:** Takes a default value as its single argument.
- ✍ **Setter Method:** Calling the setter updates React state and initiates safe UI tree updates.

```
import { useState } from 'react';

function SearchInput() {
  const [query, setQuery] = useState("");

  return (
    <input
      type="text"
      value={query}
      onChange={(e) => setQuery(e.target.value)}
    />
  );
}
```

useEffect: Side-Effect Sync

Tracking Component Lifecycles

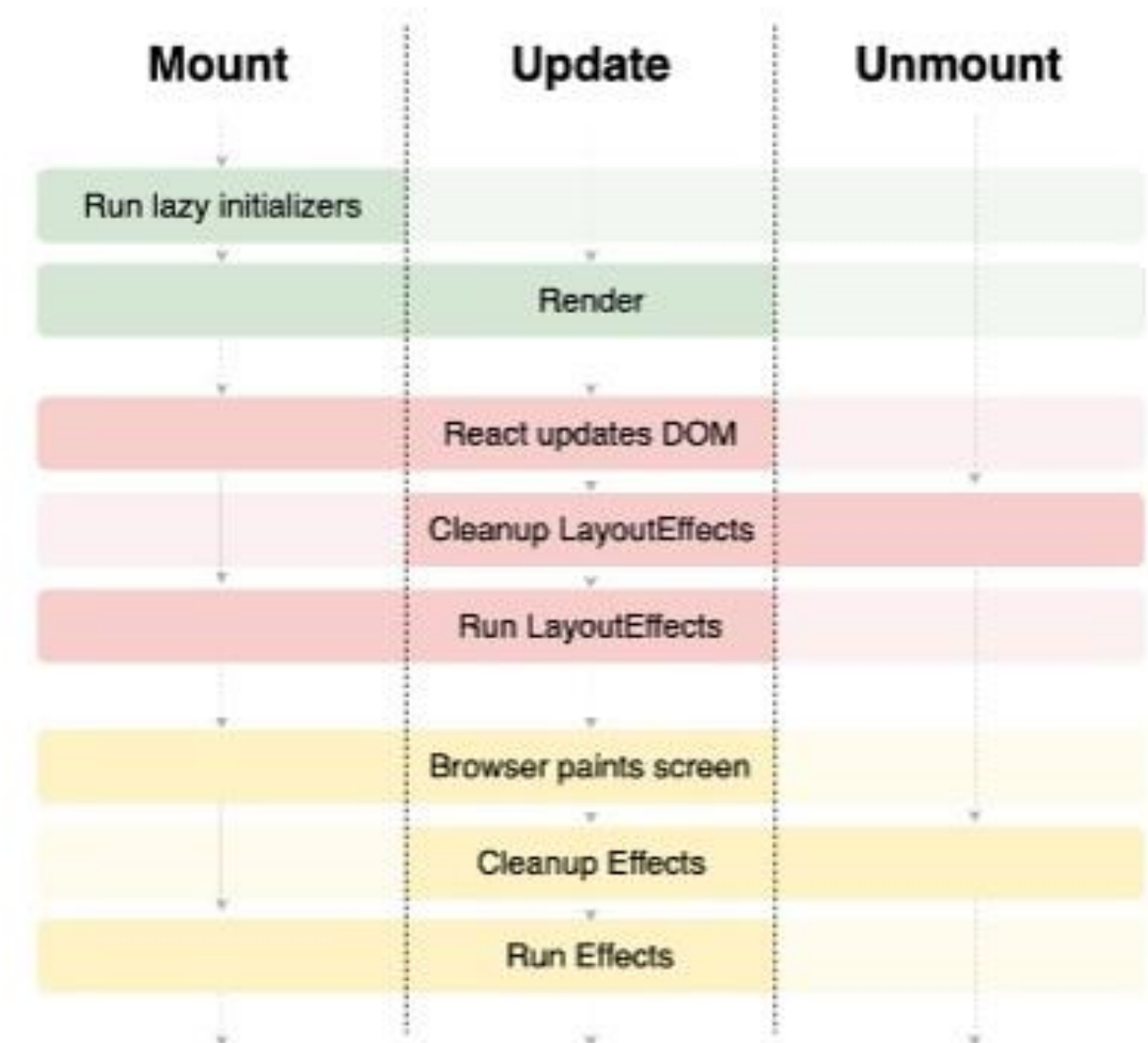
The useEffect hook binds components to external systems, side-effect dependencies, and persistent memory caches.

🔑 **Execution:** Runs after initial mounting and updates.

🔗 **Dependency Array:** Configures custom array vectors to restrict hook execution triggers.

React Hook Flow Diagram

v1.3.1 github.com/donavon/hook-flow



Notes:

1. Updates are caused by a parent re-render, state change, or context change.
2. Lazy initializers are functions passed to useState and useReducer.

PRACTICAL APPLICATION

Dynamic Search & Filters

E-commerce search indexing and price filtering workflows

Step 1: Product Search Handler



Text-based Filtering

Combine local input values with standard JavaScript string comparisons to filter through mock database items dynamically.

-  **Case Insensitivity:** Map terms using lowercase helpers for reliable key matching.
-  **Responsive Flow:** State changes propagate immediately to child components.

```
// Filter product list dynamically based on  
local query state
```

```
const filtered = products.filter(product =>  
  product.name  
    .toLowerCase()  
    .includes(searchQuery.toLowerCase())  
);
```

Step 2: Category Filter Selectors



Drop-down Categories

Leverage HTML select controls mapped directly to standard React state instances to handle strict tag-matching operations.

☰ **Multi-category logic:** Filter conditionally depending on whether 'All' or specific categories are targeted.

↶ **Unified variables:** Easily combined with existing string matching conditions.

```
// Handle fallback conditional checks
```

```
const finalProducts = filtered.filter(product => {  
  return selectedCategory === "All" ||  
    product.category === selectedCategory;  
});
```

Step 3: Multi-Criteria Price Sorters



- ◆ **Sorting Sequence Pipeline** — Sorters execute directly on shallow arrays to safely isolate and arrange prices ascending/descending.
- 📄 **Immutable Mutator Rules** — Copy reference points using spread operator `[...products]` mutating array sorts.
- ✚ **Asynchronous State Resolving** — Consistently binds to event targets, keeping component state in line with UI elements.

Session Outcome



Highly Interactive Experience

The product catalog dynamically parses search expressions, isolates category groupings, and adjusts price sorts without requesting page refreshes.

- ✓ Clean useState implementation mapping forms.
- ✓ Multi-layer dynamic arrays sorting.
- ✓ Live, real-time UI catalog updates.



Thank You